

Assignment 1

TITLE: Parallel DFS and BFS Algorithms

AIM: Design and implement Parallel Breadth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS

PRE-REQUISITES:

1. Knowledge of OPENMP programming.
2. Knowledge of parallel programming.
3. Knowledge of BFS and DFS search.

THEORY:-

What is DFS?

DFS stands for Depth-First Search. It is a popular graph traversal algorithm that explores as far as possible along each branch before backtracking. This algorithm can be used to find the shortest path between two vertices or to traverse a graph in a systematic way. The algorithm starts at the root node and explores as far as possible along each branch before backtracking. The backtracking is done to explore the next branch that has not been explored yet. DFS can be implemented using either a recursive or an iterative approach. The recursive approach is simpler to implement but can lead to a stack overflow error for very large graphs. The iterative approach uses a stack to keep track of nodes to be explored and is preferred for larger graphs. DFS can also be used to detect cycles in a graph. If a cycle exists in a graph, the DFS algorithm will eventually reach a node that has already been visited, indicating that a cycle exists.

A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

What is BFS?

BFS stands for Breadth-First Search. It is a graph traversal algorithm used to explore all the nodes of a graph or tree systematically, starting from the root node or a specified starting point, and visiting all the neighboring nodes at the current depth level before moving on to the next depth level. The algorithm uses a queue data structure to keep track of the nodes that need to be visited, and marks each visited node to avoid processing it again. The basic idea of the BFS algorithm is to visit all the nodes at a given level before moving on to the next level, which ensures that all the nodes are visited in breadth-first order.

BFS is commonly used in many applications, such as finding the shortest path between two nodes, solving puzzles, and searching through a tree or graph.

Example of DFS:

To implement DFS traversal, you need to take the following stages.

Step 1: Create a stack with the total number of vertices in the graph as the size.

Step 2: Choose any vertex as the traversal's beginning point. Push a visit to that vertex and add it to the stack.

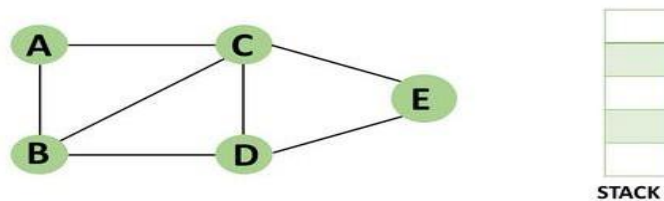
Step 3 - Push any non-visited adjacent vertices of a vertex at the top of the stack to the top of the stack.

Step 4 - Repeat steps 3 and 4 until there are no more vertices to visit from the vertex at the top of the stack.

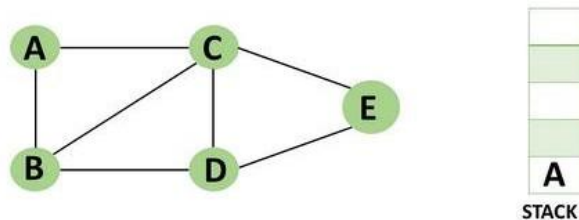
Step 5 - If there are no new vertices to visit, go back and pop one from the stack using backtracking.

Step 6 - Continue using steps 3, 4, and 5 until the stack is empty.

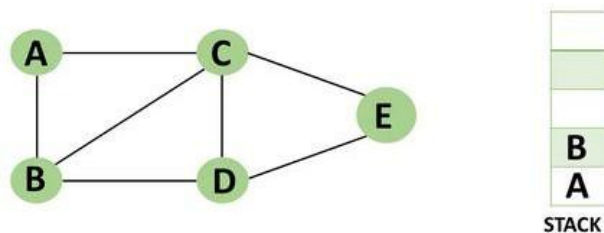
Step 7 - When the stack is entirely unoccupied, create the final spanning tree by deleting the graph's unused edges.



Step 1: Mark vertex A as a visited source node by selecting it as a source node. You should push vertex A to the top of the stack.



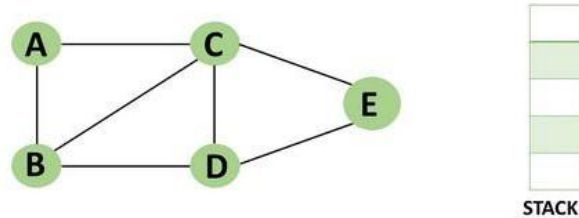
Step 2: Any nearby unvisited vertex of vertex A, say B, should be visited. You should push vertex B to the top of the stack



Step 3: From vertex C and D, visit any adjacent unvisited vertices of vertex B. Imagine you have chosen vertex C, and you want to make C a visited vertex.

.

Step 10: All of the nearby vertices of Vertex A, B, and C, have already been visited, so pop vertex A from the stack as well.



Example of BFS

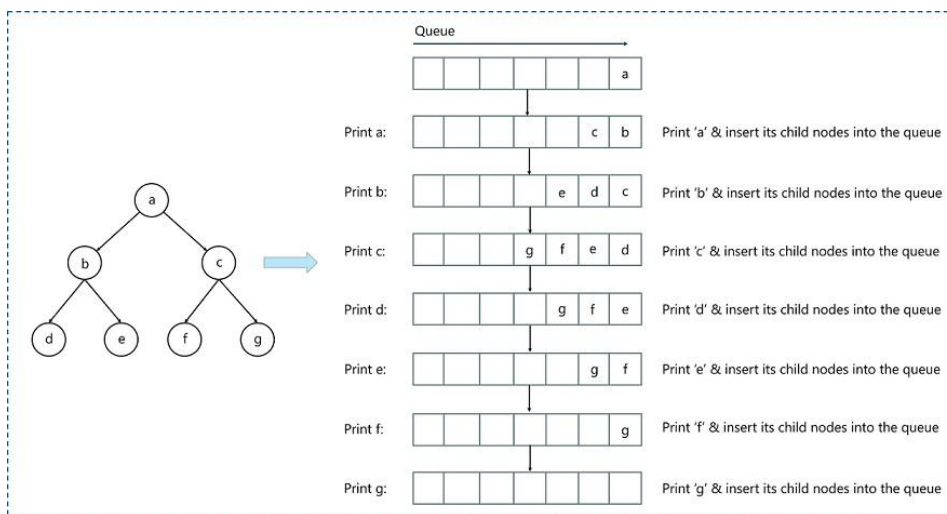
Now let's take a look at the steps involved in traversing a graph by using Breadth-First Search:

Step 1: Take an Empty Queue.

Step 2: Select a starting node (visiting a node) and insert it into the Queue.

Step 3: Provided that the Queue is not empty, extract the node from the Queue and insert its child nodes (exploring a node) into the Queue.

Step 4: Print the extracted node.



Conclusion:

Parallel BFS and DFS improve graph traversal efficiency by exploring multiple nodes simultaneously using multithreading, reducing execution time compared to sequential methods.

Input:

```
#include <iostream>
#include <vector>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V)
    };

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    // ----- DISPLAY GRAPH -----
    void displayGraph() {
        cout << "Vertices: ";
        for (int i = 0; i < V;
            i++) {
            cout << i << " ";
        }
        cout << endl;

        cout << "Edges (Adjacency List):\n";
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (int j : adj[i]) {
                cout << j << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

    // ----- PARALLEL BFS -----
    void parallelBFS(int start) {
        vector<bool> visited(V, false);
        vector<int> current_level, next_level;

        visited[start] = true;
```

```

current_level.push_back(start);

cout << "Parallel BFS: ";

while (!current_level.empty()) {
    next_level.clear();

    #pragma omp parallel for
    for (int i = 0; i < current_level.size(); i++) {
        int node = current_level[i];

        #pragma omp critical
        cout << node << " ";

        for (int neighbor : adj[node]) {
            bool should_add = false;

            #pragma omp critical
            {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    should_add = true;
                }
            }

            if (should_add) {
                #pragma omp critical
                next_level.push_back(neighbor);
            }
        }
    }

    current_level = next_level;
}

cout << endl;
}

// ----- PARALLEL DFS -----
void parallelDFSUtil(int node, vector<bool> &visited)
{
    bool alreadyVisited;

    #pragma omp critical
    {
        alreadyVisited = visited[node];
        if (!visited[node]) {
            visited[node] = true;
            cout << node << " ";
        }
    }
}

```

```

    }

    if (alreadyVisited) return;

    for (int i = 0; i < adj[node].size(); i++) {
        int neighbor = adj[node][i];

        #pragma omp task
        parallelDFSUtil(neighbor, visited);
    }
}

void parallelDFS(int start) {
    vector<bool> visited(V, false);

    cout << "Parallel DFS: ";

    #pragma omp parallel
    {
        #pragma omp single
        {
            parallelDFSUtil(start, visited);
        }
    }

    cout << endl;
}

};

// -----
// MAI
N int main() {
    Graph g(7);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(1, 4);
    g.addEdge(2, 5);
    g.addEdge(2, 6);

    // Display graph
    g.displayGraph();

    // Traversals
    g.parallelBFS(0);
    g.parallelDFS(0);

    return 0;
}

```

Output:

```
[projectlab@fedora ~]$ g++ -fopenmp BFS_DFS.cpp -o graph
[projectlab@fedora ~]$ ./graph
Parallel BFS: 0 2 1 5 3 4 6
Parallel DFS: 0 1 2 3 4 5 6
[projectlab@fedora ~]$ g++ -fopenmp BFS_DFS.cpp -o graph
./graph
Vertices: 0 1 2 3 4 5 6
Edges (Adjacency List):
0 -> 1 2
1 -> 0 3 4
2 -> 0 5 6
3 -> 1
4 -> 1
5 -> 2
6 -> 2

Parallel BFS: 0 1 2 3 4 5 6
Parallel DFS: 0 1 2 3 4 5 6
[projectlab@fedora ~]$
```



DHOLE PATIL EDUCATION SOCIETY's

DHOLE PATIL COLLEGE OF ENGINEERING

Accredited by NAAC with A+ Grade, An ISO 9001:2015 Certified Institute

1284, Near Eon IT Park Kharadi, Dhole Patil College Road, Wagholi, Pune-412207

Website: www.dpcoepune.edu.in E-mail: dpcoepune@gmail.com, Phone: 020-

Assignment 2

TITLE: Parallel Sorting Algorithms

AIM: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

PRE-REQUISITES:

1. Knowledge of OPENMP programming.
2. Knowledge of parallel programming.
3. Knowledge of merge sort and merge sort method.

THEORY:-

OpenMP is a set of compiler directives as well as an API for programs written in C, C++, or FORTRAN that provides support for parallel programming in shared-memory environments. OpenMP identifies parallel regions as blocks of code that may run in parallel. Application developers insert compiler directives into their code at parallel regions, and these directives instruct the OpenMP run-time library to execute the region in parallel. When OpenMP encounters the directive

#pragma omp parallel

It creates as many threads which are processing cores in the system. Thus, for a dual-core system, two threads are created, for a quad-core system, four are created; and so forth. Then all the threads simultaneously execute the parallel region. When each thread exits the parallel region, it is terminated. OpenMP provides several additional directives for running code regions in parallel, including parallelizing loops.

Conclusion:

Parallel Bubble Sort and Merge Sort enhance sorting performance by dividing the data into smaller parts and processing them concurrently, leading to better scalability and faster execution on multi-core systems.

Input:

```
#include <iostream>

#include <vector>

#include <omp.h>

using namespace std;

// ----- UTILITY-----

void printArray(vector<int> &arr) {
    for (int x : arr) cout << x << " ";
    cout << endl;
}

// ----- SEQUENTIAL BUBBLE SORT -----

void bubbleSortSequential(vector<int> &arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1;
              j++) if (arr[j] > arr[j + 1])
                    swap(arr[j], arr[j + 1]);
}

// ----- PARALLEL BUBBLE SORT -----

void bubbleSortParallel(vector<int> &arr) {
    int n = arr.size();
    for (int i = 0; i < n; i++) {
```

```

// Even phase

#pragma omp parallel for
for (int j = 0; j < n - 1; j +=
2)
    if (arr[j] > arr[j + 1])
        swap(arr[j], arr[j + 1]);

// Odd phase

#pragma omp parallel for
for (int j = 1; j < n - 1; j +=
2)
    if (arr[j] > arr[j + 1])
        swap(arr[j], arr[j + 1]);
}
}

// ----- MERGE FUNCTION -----

void merge(vector<int> &arr, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int i = 0; i < k; i++) arr[left + i] = temp[i];
}

```

```
// ----- SEQUENTIAL MERGE SORT -----
```

```
void mergeSortSequential(vector<int> &arr, int left, int right) {  
    if (left >= right) return;  
    int mid = (left + right) / 2;  
    mergeSortSequential(arr, left, mid);  
    mergeSortSequential(arr, mid + 1, right);  
    merge(arr, left, mid, right);  
}
```

```
// ----- PARALLEL MERGE SORT-----
```

```
void mergeSortParallel(vector<int> &arr, int left, int right) {  
    if (left >= right) return;  
  
    if (right - left < 1000) { // small arrays handled sequentially  
        mergeSortSequential(arr, left, right);  
        return;  
    }
```

```
    int mid = (left + right) / 2;
```

```
    #pragma omp task shared(arr)
```

```
    mergeSortParallel(arr, left, mid);
```

```
    #pragma omp task shared(arr)
```

```
    mergeSortParallel(arr, mid + 1, right);
```

```

#pragma omp taskwait
merge(arr, left, mid, right);
}

// ----- MAIN -----

int main() {
    int n;
    cout << "Enter size of the array: ";
    cin >> n;

    vector<int> arr(n), arr1, arr2, arr3, arr4;

    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    arr1 = arr2 = arr3 = arr4 = arr;

    double start, end;

    // ----- BUBBLE SORT -----

    start = omp_get_wtime();
    bubbleSortSequential(arr1);
    end = omp_get_wtime();

```

```
cout << "Sequential Bubble Sort Time: " << (end - start) << " sec\n";
```

```
start = omp_get_wtime();
```

```
bubbleSortParallel(arr2);
```

```
end = omp_get_wtime();
```

```
cout << "Parallel Bubble Sort Time: " << (end - start) << " sec\n";
```

```
// ----- MERGE SORT -----
```

```
start = omp_get_wtime();
```

```
mergeSortSequential(arr3, 0, n - 1);
```

```
end = omp_get_wtime();
```

```
cout << "Sequential Merge Sort Time: " << (end - start) << " sec\n";
```

```
start = omp_get_wtime();
```

```
#pragma omp parallel
```

```
{
```

```
    #pragma omp single
```

```
    mergeSortParallel(arr4, 0, n - 1);
```

```
}
```

```
end = omp_get_wtime();
```

```
cout << "Parallel Merge Sort Time: " << (end - start) << " sec\n";
```

```
// Optional: display sorted array for small inputs
```

```
if (n <= 20) {
```

```
    cout << "Sorted Array (Bubble Sort Parallel): ";
```

```
    printArray(arr2);
```

```
        cout << "Sorted Array (Merge Sort Parallel): ";  
        printArray(arr4);  
    }  
  
    return 0;  
}
```

Output:

```
[projectlab@fedora ~]$ g++ -fopenmp hpc2.cpp -o graph
```

```
[projectlab@fedora ~]$ ./graph
```

```
Enter size of the array: 5
```

```
Enter 5 elements:
```

```
1 2 3 4 5
```

```
Sequential Bubble Sort Time: 8.67e-07 sec
```

```
Parallel Bubble Sort Time: 0.00036331 sec
```

```
Sequential Merge Sort Time: 5.042e-06 sec
```

```
Parallel Merge Sort Time: 6.1761e-05 sec
```

```
Sorted Array (Bubble Sort Parallel): 1 2 3 4 5
```

```
Sorted Array (Merge Sort Parallel): 1 2 3 4 5
```



DHOLE PATIL EDUCATION SOCIETY'S

DHOLE PATIL COLLEGE OF ENGINEERING

Accredited by NAAC with A+ Grade, An ISO 9001:2015 Certified Institute

1284, Near Eon IT Park Kharadi, Dhole Patil College Road, Wagholi, Pune-412207

Website: www.dpcoepune.edu.in E-mail: dpcoepune@gmail.com, Phone: 020-

Assignment 3

TITLE: Parallel Reduction Implementation

AIM: Implement Parallel Reduction using Min, Max, Sum and Average operations.

OBJECTIVES:

Students should be able to perform Parallel Reduction design pattern to perform fundamental statistical operations—Minimum, Maximum, Sum, and Average.

PRE-REQUISITES:

1. Knowledge of OPENMP programming.
2. Knowledge of parallel programming.
3. Knowledge of Min, Max, Sum and Average operations.

Reduction is a process where a set of input values is combined using a binary associative operator (like addition or multiplication) to produce a single scalar result.

The Problem: Race Conditions

In a standard parallel loop, if multiple threads try to update a single shared variable (e.g., `sum = sum + array[i]`) at the same time, the result becomes unpredictable due to **race conditions**.

The Solution: Parallel Reduction Pattern

1. **Local Accumulation:** Each thread creates a private copy of the reduction variable and calculates a "local" result for its portion of the data.
2. **Global Synchronization:** Once all threads finish, their local results are combined into the final global variable using the specified operator.
 3. **Sum (+):** Combines elements by addition. The identity value is 0.
 4. **Min (min):** Finds the smallest value. The identity value is INT_MAX.
 5. **Max (max):** Finds the largest value. The identity value is INT_MIN.
 6. **Average:** This is a **derived operation**. It is calculated by performing a parallel reduction for the **Sum**, then dividing that total by the count of elements in a sequential step.

Conclusion:

Parallel reduction efficiently performs aggregate operations like sum, minimum, maximum, and average by combining partial results from multiple threads while avoiding race conditions.

Input:

```
#include <iostream>
#include <vector>
#include <omp.h>
using namespace std;

int main() {
    int n;
    cout << "Enter size of the array: ";
    cin >> n;

    vector<int> arr(n);
    cout << "Enter " << n << " elements (space-separated):\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int    minVal    =
    arr[0]; int maxVal
    = arr[0]; long long
    sum = 0;

    // Parallel reduction
    #pragma omp parallel for reduction(min:minVal) reduction(max:maxVal) reduction(+:sum)
    for (int i = 0; i < n; i++) {
        if (arr[i] < minVal) minVal = arr[i]; // min
        reduction if (arr[i] > maxVal) maxVal = arr[i]; //
        max reduction sum += arr[i];    // sum reduction
    }

    double average = static_cast<double>(sum) / n;

    cout << "Minimum: " << minVal <<
    endl; cout << "Maximum: " << maxVal
    << endl; cout << "Sum: " << sum <<
    endl;
    cout << "Average: " << average <<
    endl; return 0;
}
```


Output:

```
[projectlab@fedora ~]$ g++ -fopenmp hpc3.cpp -o graph
[projectlab@fedora ~]$ ./graph
Enter size of the array: 5
Enter 5 elements (space-separated):
1 2 3 4 5
Minimum: 1
Maximum: 5
Sum: 15
Average: 3
[projectlab@fedora ~]$
```